



Федеральное государственное автономное образовательное  
учреждение высшего образования  
«Национальный исследовательский университет ИТМО»

---

ФАКУЛЬТЕТ «Программной инженерии и компьютерной техники»

---

ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА «Проектирование и разработка систем больших данных»

---

НАПРАВЛЕНИЕ ПОДГОТОВКИ (СПЕЦИАЛЬНОСТЬ) «09.04.04 Программная инженерия»

---

Программной инженерии и компьютерной техники (ФПИ и КТ)

## ОТЧЕТ

по лабораторной работе №1

по курсу «Структуры и алгоритмы в базах данных и распределенных  
системах»

на тему: «Реализация алгоритмов хэширования (perfect, extendible, min)»

Обучающийся: Постнов С. А., Р4135  
(Ф.И.О, № группы)

Преподаватель Платонов А. В., доцент  
(Ф.И.О, должность)

Преподаватель Портнов П. В., ассистент  
(Ф.И.О, должность)

2026 г.

# СОДЕРЖАНИЕ

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>1</b> | <b>Теоретическая часть</b>                    | <b>3</b>  |
| 1.1      | Perfect hashing . . . . .                     | 3         |
| 1.2      | Extendible hashing . . . . .                  | 3         |
| <b>2</b> | <b>Практическая часть</b>                     | <b>5</b>  |
| 2.1      | Реализация perfect hash таблицы . . . . .     | 5         |
| 2.2      | Реализация extendible hash таблицы . . . . .  | 5         |
| <b>3</b> | <b>Исследовательская часть</b>                | <b>7</b>  |
| 3.1      | Методика замеров perfect hashing . . . . .    | 7         |
| 3.2      | Методика замеров extendible hashing . . . . . | 7         |
| 3.3      | Результаты . . . . .                          | 7         |
| 3.4      | Результаты для extendible hashing . . . . .   | 9         |
| <b>4</b> | <b>Вывод</b>                                  | <b>13</b> |

# 1 Теоретическая часть

## 1.1 Perfect hashing

Задача: хранить пары ключ–значение так, чтобы операции `set/get` выполнялись за  $O(1)$  (в среднем) и не возникало коллизий для фиксированного набора ключей.

*Perfect hash* для множества ключей  $S$  — это хэш–функция  $h$ , которая отображает ключи из  $S$  в индексы таблицы без коллизий. В классическом варианте (FKS) используется двухуровневая схема: первичная таблица распределяет ключи по корзинам, а для каждой корзины строится вторичная таблица, в которой подбираются параметры так, чтобы коллизий не было.

В данной работе реализована схожая идея:

- первичная таблица имеет размер, равный ближайшему простому  $\geq n$ ;
- в ячейке хранится либо одна пара `key/value`, либо указатель на вторичную таблицу;
- при коллизии создаётся/перестраивается вторичная таблица: увеличивается её размер и перебираются различные хэш–функции до получения раскладки без коллизий.

## 1.2 Extendible hashing

*Extendible hashing* предназначен для динамических наборов данных, где число ключей заранее неизвестно и структура должна расти и уменьшаться без полной перестройки таблицы. В отличие от perfect hashing, здесь допускаются коллизии, но они локализуются в бакетах фиксированной ёмкости.

Основные элементы алгоритма:

- директория содержит  $2^d$  указателей на бакеты, где  $d$  — глобальная глубина;
- каждый бакет имеет локальную глубину, определяющую, по скольким младшим битам хэша он адресуется;
- при переполнении бакет расщепляется, а при необходимости директория удваивается;

- при удалении возможны обратные операции: слияние соседних бакетов и уменьшение директории.

Такая схема обеспечивает амортизированно быстрые операции `insert/get/update/delete` и хорошо подходит для внешней памяти: бакеты можно размещать непосредственно в файле и адресовать через отображение файла в память.

## 2 Практическая часть

### 2.1 Реализация perfect hash таблицы

Публичный интерфейс реализован в файлах `inc/perfect.h` и `src/perfect.c`:

- `create_perfect_table` создаёт таблицу нужного размера;
- `perfect_set` добавляет/обновляет пару ключ–значение;
- `perfect_get` выполняет поиск значения по ключу;
- `free_perfect_table` освобождает память.

Структура `perfect_hash_table_t` содержит массив указателей на `ceil_value_t`. Каждая ячейка либо хранит одну пару `key/value`, либо указывает на вторичную таблицу (вложенный `perfect_hash_table_t`).

Для хэширования строк используются функции из `inc/hash.h`. При перестроении вторичных таблиц перебирается набор хэш–функций до построения раскладки без коллизий.

### 2.2 Реализация extendible hash таблицы

Реализация extendible hashing вынесена в каталог `extendible/`. Основной код расположен в файле `extendible/extendible.go`. Таблица хранится не в оперативной памяти как отдельная структура с последующей сериализацией, а непосредственно в файле, отображённом в память с помощью `mmap`.

Файл разбит на три логические области:

- 1) заголовок с метаданными таблицы (магическое число, версия формата, глобальная глубина, размер бакета);
- 2) директория фиксированного максимального размера, содержащая идентификаторы бакетов;
- 3) область бакетов, где для каждого бакета хранятся локальная глубина, число записей и массив пар `key/value/hash`.

Публичный интерфейс включает операции `Insert`, `Update`, `Delete`, `Get`, `Put`, а также открытие и закрытие таблицы в файле. При переполнении вызывается `split` бакета, при удалении — `merge buddy`-бакетов и, если возможно, `shrink` директории.

Для проверки корректности использованы `unit`- и `fuzz`-тесты, а для замеров — `benchmark tests` на Go 1.25 с использованием `b.Loop()`.

## 3 Исследовательская часть

### 3.1 Методика замеров perfect hashing

Для замеров использована программа `src/benchmark.c`. Для каждого датасета из `data/bench/manifest.txt`:

- 1) создаётся пустая perfect hash таблица;
- 2) измеряется время вставки всех  $n$  ключей (`perfect_set`);
- 3) измеряется время чтения всех  $n$  ключей (`perfect_get`).

Каждый прогон усредняется по `BENCH_ITERS` (по умолчанию 5).

### 3.2 Методика замеров extendible hashing

Для extendible hashing использованы встроенные benchmark-тесты Go и отдельная утилита профилирования из каталога `extendible/cmd/profile`. Измерялись четыре операции: `insert`, `update`, `delete`, `get`.

Для каждой мощности набора данных выполнялись:

- 1) бенчмарки со считыванием времени на операцию, пропускной способности, количества аллокаций и объёма выделенной памяти;
- 2) отдельный профилируемый прогон с замером wall-clock времени, CPU user/sys time, суммарного объёма аллокаций и общего числа аллокаций.

Исходные данные сохраняются в `extendible/metrics/raw/*.csv`, а графики строятся автоматически в `extendible/metrics/plots/*.pdf`.

### 3.3 Результаты

Таблица 3.1 содержит усреднённые значения задержек и пропускной способности.

Графики результатов замеров представлены на рисунках 3.1 и 3.2:

Таблица 3.1 – Результаты бенчмарка perfect hash (среднее по итерациям)

| $N$     | вставка,<br>нс/оп | вставка, млн<br>оп/с | поиск, нс/оп | поиск, млн<br>оп/с |
|---------|-------------------|----------------------|--------------|--------------------|
| 50      | 344.0             | 2.98                 | 80.0         | 12.50              |
| 250     | 528.0             | 1.90                 | 112.0        | 8.95               |
| 1250    | 226.4             | 4.42                 | 93.0         | 10.76              |
| 7500    | 314.9             | 3.24                 | 82.8         | 12.19              |
| 20000   | 404.5             | 2.51                 | 66.2         | 15.22              |
| 50000   | 217.0             | 4.61                 | 52.4         | 19.09              |
| 100000  | 127.7             | 7.83                 | 43.2         | 23.15              |
| 250000  | 225.7             | 4.43                 | 58.3         | 17.14              |
| 500000  | 141.3             | 7.08                 | 52.2         | 19.15              |
| 750000  | 138.7             | 7.21                 | 52.0         | 19.24              |
| 1000000 | 197.4             | 5.07                 | 57.9         | 17.29              |
| 2000000 | 164.9             | 6.07                 | 57.2         | 17.51              |
| 5000000 | 194.1             | 5.15                 | 81.2         | 12.32              |

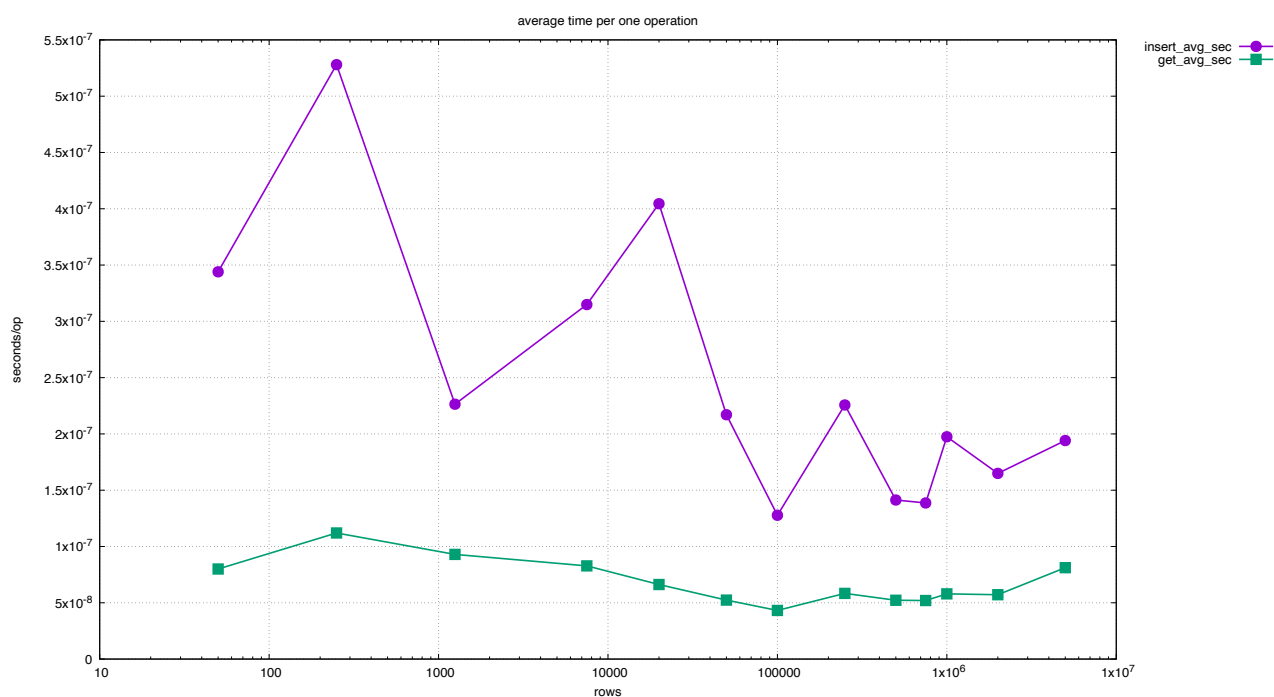


Рисунок 3.1 – Средняя задержка операций вставки/поиска для perfect hash



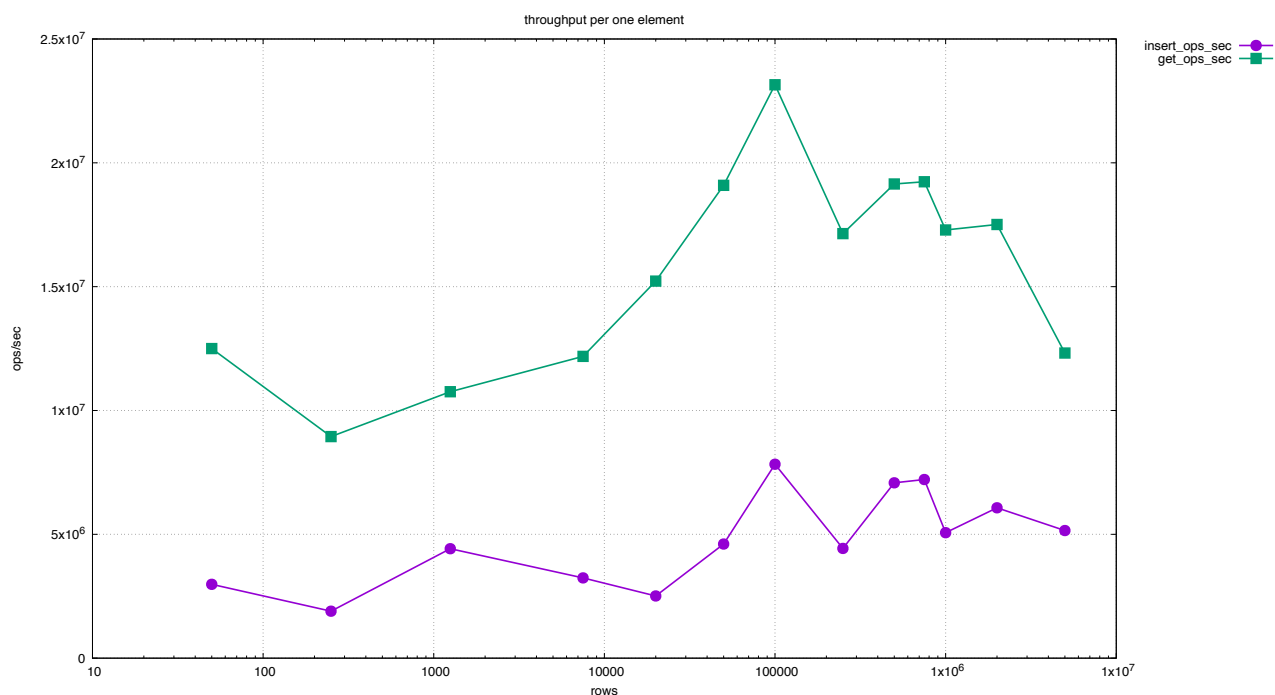


Рисунок 3.2 – Пропускная способность операций вставки/поиска для perfect hash

### 3.4 Результаты для extendible hashing

Таблицы 3.2 и 3.3 содержат результаты бенчмарка extendible hashing для всех измеренных размеров набора данных.

Таблица 3.2 – Результаты бенчмарка extendible hashing: задержка, нс/элемент

| $N$      | вставка | обновление | удаление | поиск |
|----------|---------|------------|----------|-------|
| 1000     | 2367.0  | 773.2      | 823.3    | 781.1 |
| 5000     | 691.5   | 179.7      | 162.8    | 154.1 |
| 10000    | 239.7   | 122.2      | 127.8    | 132.8 |
| 50000    | 90.0    | 72.9       | 86.5     | 83.5  |
| 100000   | 83.0    | 52.2       | 107.0    | 93.4  |
| 500000   | 144.5   | 42.4       | 238.1    | 43.0  |
| 1000000  | 219.7   | 58.2       | 444.6    | 59.3  |
| 2500000  | 588.2   | 105.7      | 992.0    | 104.7 |
| 5000000  | 1080.0  | 124.2      | 3056.0   | 125.2 |
| 10000000 | 2126.0  | 137.5      | 10252.0  | 137.5 |

Таблица 3.3 – Результаты бенчмарка extendible hashing: пропускная способность, млн оп/с

| $N$      | вставка | обновление | удаление | поиск |
|----------|---------|------------|----------|-------|
| 1000     | 0.42    | 1.29       | 1.21     | 1.28  |
| 5000     | 1.45    | 5.56       | 6.14     | 6.49  |
| 10000    | 4.17    | 8.18       | 7.82     | 7.53  |
| 50000    | 11.11   | 13.72      | 11.57    | 11.97 |
| 100000   | 12.05   | 19.14      | 9.35     | 10.70 |
| 500000   | 6.92    | 23.60      | 4.20     | 23.26 |
| 1000000  | 4.55    | 17.19      | 2.25     | 16.88 |
| 2500000  | 1.70    | 9.46       | 1.01     | 9.55  |
| 5000000  | 0.93    | 8.05       | 0.33     | 7.99  |
| 10000000 | 0.47    | 7.27       | 0.10     | 7.27  |

Из таблиц видно, что операции **get** и **update** наиболее стабильны по мере роста набора данных. Операция **insert** заметно дороже из-за split бакетов и расширения директории, а **delete** оказывается самой тяжёлой на больших  $N$  из-за merge/shrink после удаления элементов.

Дополнительные профилировочные замеры показывают, что основной объём аллокаций приходится на **insert**; для **update**, **delete** и **get** объём дополнительных аллокаций почти не растёт и определяется в основном накладными расходами рантайма.

Графики результатов замеров extendible hashing представлены на рисунках ниже.

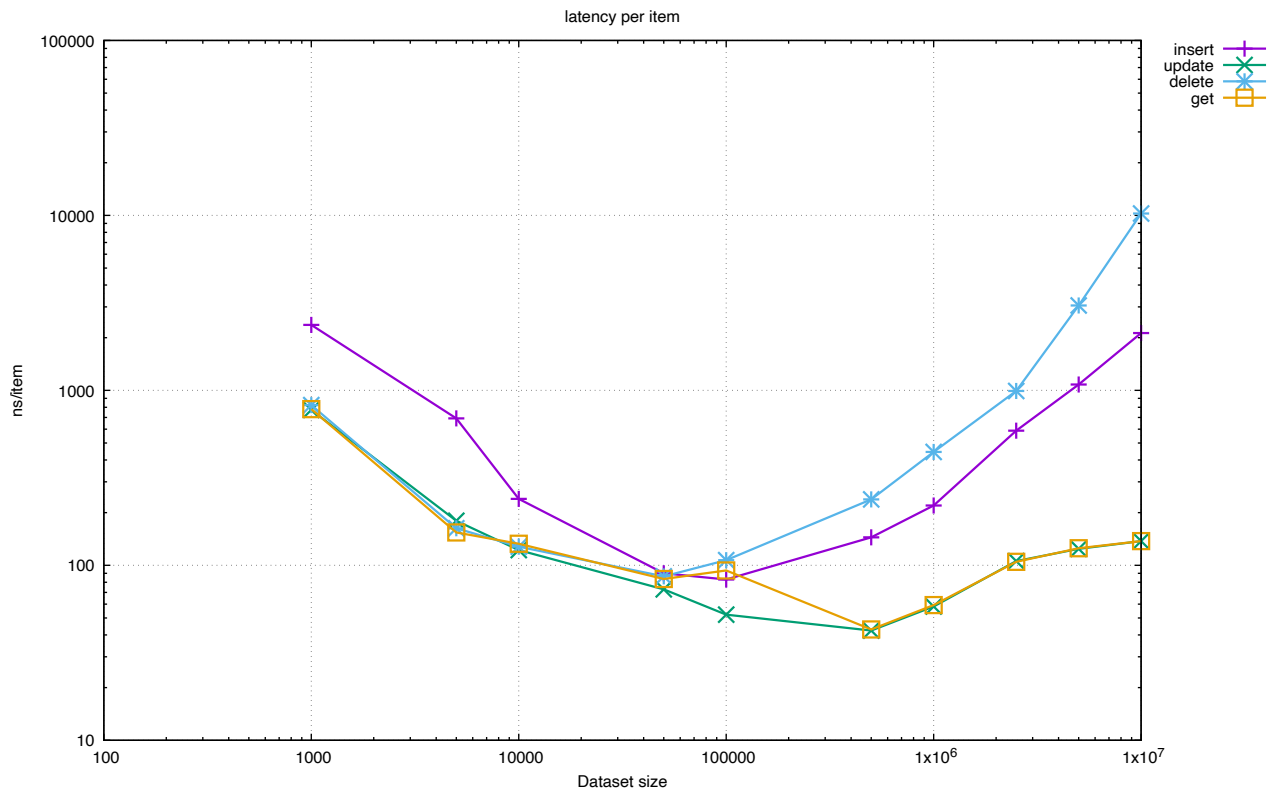


Рисунок 3.3 – Задержка операций extendible hashing на элемент

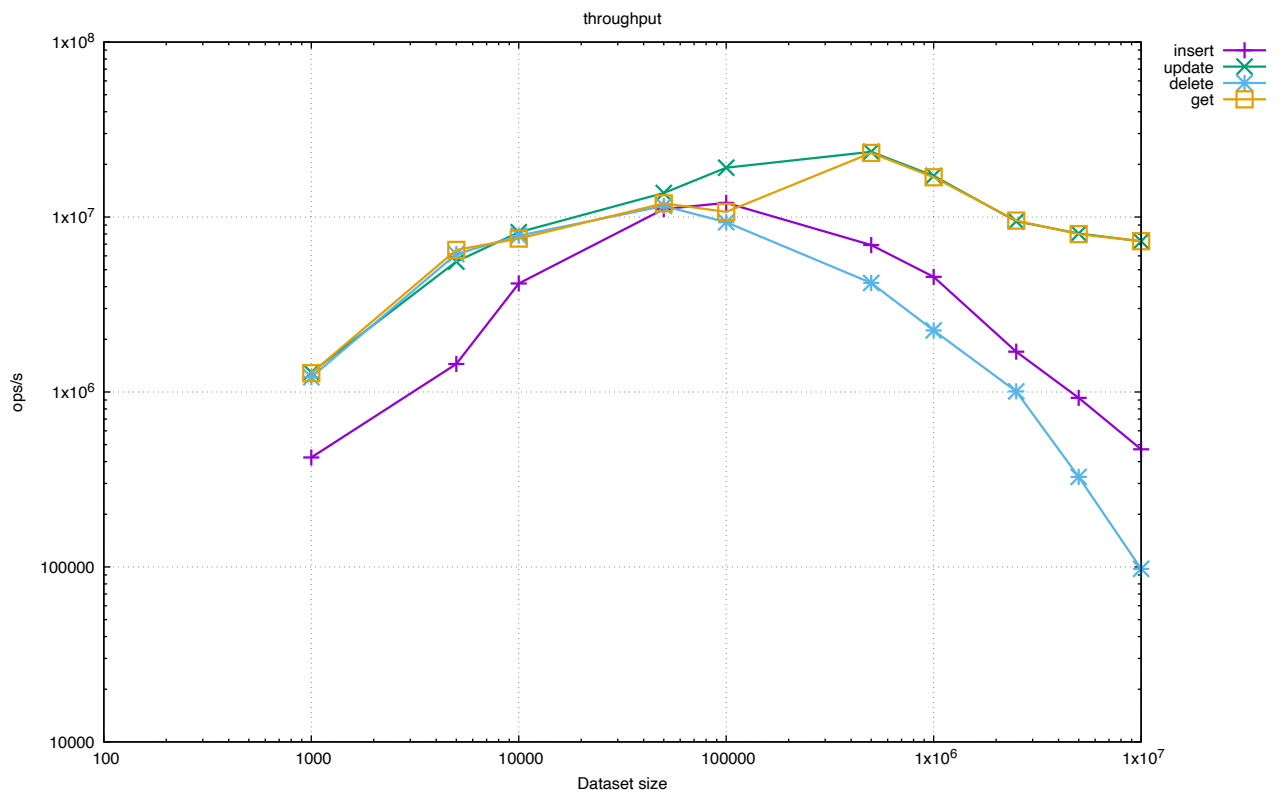


Рисунок 3.4 – Пропускная способность операций extendible hashing

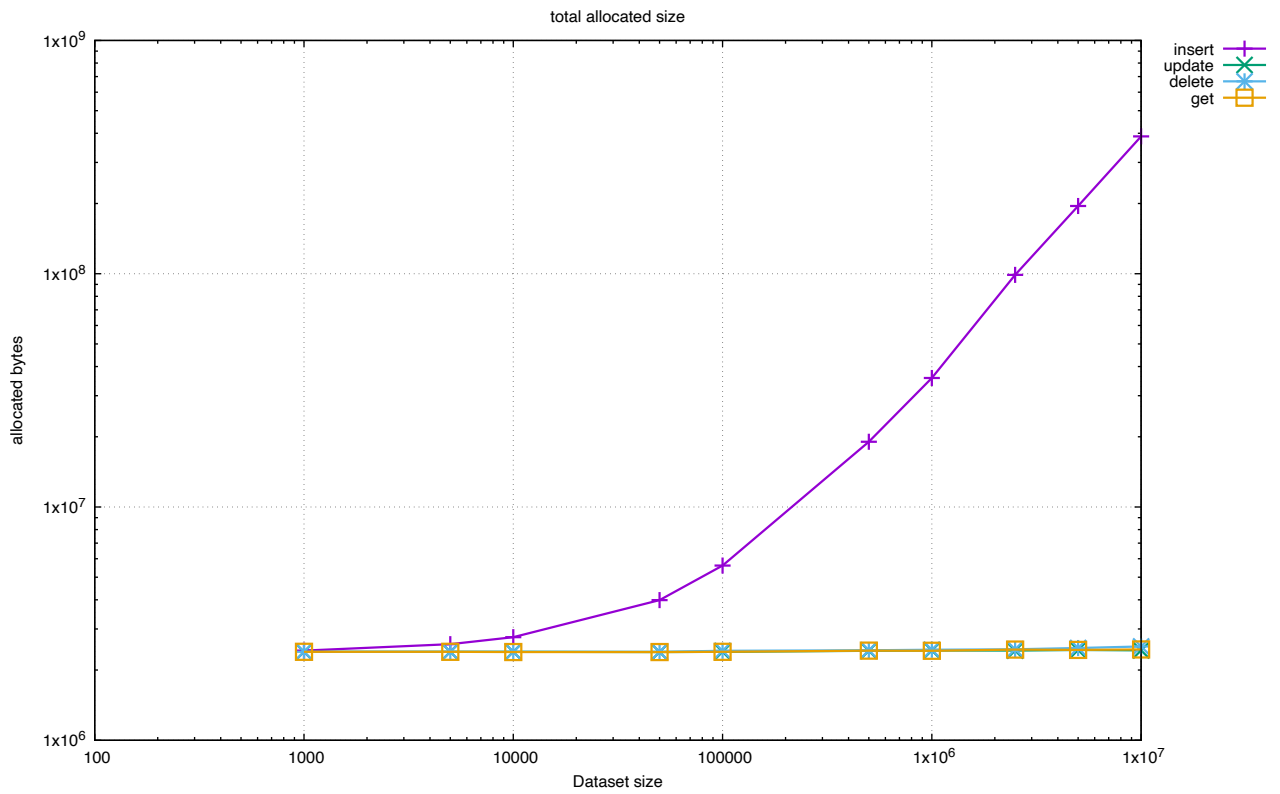


Рисунок 3.5 – Суммарный размер аллокаций для extendible hashing

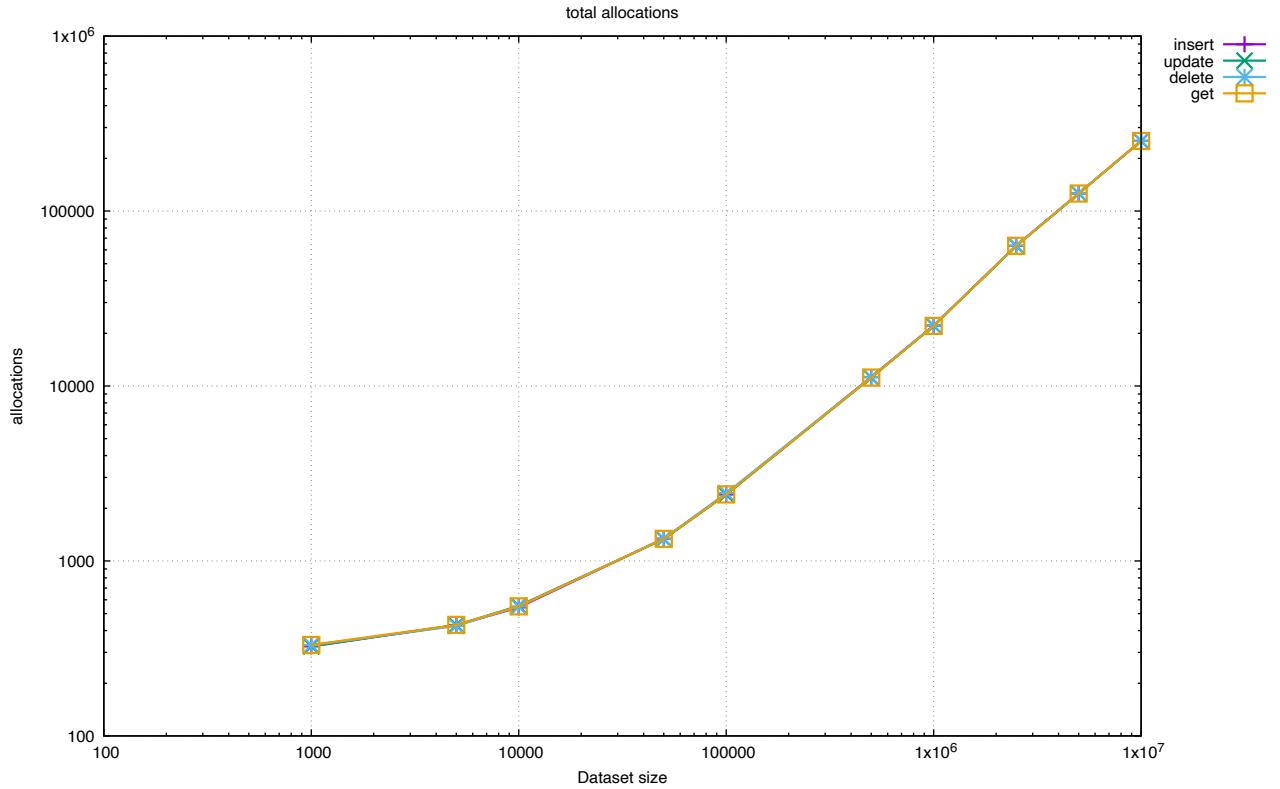


Рисунок 3.6 – Общее число аллокаций для extendible hashing

## 4 Вывод

Реализована perfect hash таблица для строковых ключей с поддержкой операций вставки и поиска. Также реализована extendible hash таблица с файловым хранением на базе `mmap`, поддерживающая динамические операции вставки, чтения, обновления и удаления.

Проведены замеры производительности на наборах данных разного размера. Для perfect hashing подтверждена эффективность на статическом множестве ключей, а для extendible hashing показано, что операции `get` и `update` остаются самыми быстрыми, тогда как `insert` и особенно `delete` несут дополнительные издержки на модификацию структуры каталогов и бакетов. Результаты представлены в таблицах и на графиках.